

Assignment 2

Topic: Custom System Calls

Autumn 2026-27

Submission Deadline: 28th Aug, EOD

Course: Advances in OS Design

Total Marks: 100

Steps to Add System Calls Inside the Kernel

1. Create a new directory (e.g., `custom_syscalls/`) inside the root of the kernel source tree.
2. Place all source code and the local `Makefile` inside this directory.
3. Open the top-level `Kbuild` file of the kernel. Scroll to the bottom and add the directory name: `obj-y += custom_syscalls/`.
4. Open the `include/linux/syscalls.h` file and add a line (an example is shown below) at the end for the syscall before `#endif`. You need to prefix the syscall name with “sys” as shown:

```
asmlinkage long sys_proc_info(pid_t pid, unsigned int flags,
char __user *buffer, size_t size);
```

5. Open `arch/x86/entry/syscalls/syscall_64.tbl` and register the new system calls at free entry numbers (e.g., starting from 451).
6. Compile the kernel, install modules, reboot into the modified kernel.
7. Implement wrappers around the custom system calls to make them usable by user-space C programs. Use the `syscall()` function provided by glibc.

Submission Instructions

1. Your code must compile on Linux kernel version **6.1.6** as mentioned in the setup guide.
2. Submit a single **ZIP** file (mention your group number) which will have 2 separate folders for two tasks in this assignment.
3. Each folder should have custom system call implementation files and all kernel files which are modified to use this system calls from user space.
4. The folder should also have C library wrapper files and the user-space test programs with `Makefile` in a different sub-directory from the kernel code.
5. A `README` file in each task detailing all modified files, instructions on how to use the library, and any additional design details.
6. A testing report (1-2 pages) detailing how you verified the correctness of your system calls. For example:

```
submission.zip
|
+-- Task_A/
|   +-- kernel/
|   |   +-- custom_syscalls/
|   |   +-- modified_kernel_files/
|   +-- userspace/
|       +-- wrappers/
|       +-- tests/
|
+-- Task_B/
    +-- (same structure as Task_A)
```

Task A: Process Information

Requirement

The requirement for this task is to design a custom system call named **proc_info**. When process-specific mode is selected, the system call shall return information about one chosen process. When system-wide mode is selected, the system call shall return a summary of scheduler and process activity for the entire system.

The implementation must validate the supplied flags before performing any kernel work. The interface must also remain safe for communication between kernel space and user space, which means the kernel must copy the final formatted result into a user-provided buffer and must not expose kernel memory directly.

- **Process-specific mode**
 - Parent Process ID
 - State as a numeric value
 - Effective Priority
 - Scheduling class as a String
 - Number of Child Processes
 - Number of Sibling Processes
- **System-wide mode**
 - Total Number of Processes
 - Number of Processes in `TASK_RUNNING` State
 - Number of Processes in `TASK_INTERRUPTIBLE` State
 - Number of Processes in `TASK_UNINTERRUPTIBLE` State
 - Number of Processes in RT Class
 - Number of Processes in Fair Class
 - Number of Processes in CFS Runqueue
 - PID of the Process with Minimum Vruntime Value
 - Minimum Vruntime Value
 - Total Load on CFS Runqueue

Arguments

- **pid**: PID of the process in process-specific mode; set to 0 in system-wide mode.
- **flags**: mode selector as an unsigned integer.
 - Process-specific mode: `PROC_INFO_PID`
 - System-wide mode: `PROC_INFO_SYSTEM`
- **buffer**: a char buffer that will store the extracted fields. You have to store all the fields that are extracted in this buffer, and then parse the buffer in the wrapper function. The buffer should contain the fields in the above mentioned order.
- **size**: size of the user-space buffer.

Return Values

- **0**: returned when the system call completes successfully and the requested fields are copied into **buffer**.
- **-EINVAL**: returned when **flags** contains unsupported bits, when more than one mode is selected, or when the supplied arguments are invalid.

- **-ESRCH**: returned when process-specific mode is selected and no process exists for the supplied `pid`.
- **-EFAULT**: returned when `buffer` is not a valid user-space address or the kernel cannot copy data to it.
- **-ENOSPC**: returned when `size` is too small to hold all fields in the required order.

Task B: Context Switch Tracker

Requirement

The requirement for this task is to design three custom system calls named **sys_register**, **sys_fetch**, and **sys_deregister**. These system calls shall maintain a list of monitored processes and track the cumulative number of voluntary and involuntary context switch events suffered by them and their threads.

The monitored processes shall be maintained using a doubly linked list based on the Linux kernel list mechanism provided by `include/linux/list.h` and should statically define the head of the list. Each entry in the list shall store the PID of the monitored process and the context switch statistics of that process (*struct pid_node*). The implementation shall also define a context switch structure containing the total voluntary and involuntary context switches across all monitored processes (*struct pid_ctxt_switch*). This structure should be defined in a header file since it is required in both kernel space and userspace.

Data Structure

```
struct pid_node {
    pid_t pid;
    unsigned long ninvctxt;
    unsigned long nvctxt;
    struct list_head next_prev_list;
};

struct pid_ctxt_switch {
    unsigned long tot_invctxt;
    unsigned long tot_vctxt;
};
```

System Calls

- **int sys_register(pid_t pid)**: adds the supplied PID to the tail of the monitored process list. The system call shall verify that the PID is valid and that the corresponding process exists before adding it to the list.
- **int sys_fetch(struct pid_ctxt_switch *stats)**: iterates through the monitored process list and obtains the cumulative number of voluntary and involuntary context switch events. The resulting values shall be copied into the user-provided **stats** structure.
- **int sys_deregister(pid_t pid)**: searches the monitored process list for the supplied PID and removes the corresponding node from the list.

Return Values

- **0**: returned when the requested system call completes successfully.
- **-EINVAL**: returned when **pid** is less than 1.
- **-ESRCH**: returned by **sys_register** when no process exists for the supplied **pid**.
- **-ESRCH**: returned by **sys_deregister** when the supplied **pid** is not present in the monitored process list.
- **-EFAULT**: returned by **sys_fetch** when **stats** is not a valid user-space address or the kernel cannot copy the statistics to it.
- **-ENOSPC**: returned by **sys_register** when sufficient kernel memory is not available to create a new PID node.